Deploy RDS in a 3-Tier VPC

Hands-On Lab · Web, App, and Database Tiers

1. Lab Objective

In a basic RDS lab you connect to your database straight from your laptop, which is fine for learning but not how real systems are built. In this lab you'll deploy RDS the way production systems do: inside a **3-tier VPC** where the database is completely hidden from the internet, and only the application tier can talk to it.

You'll build a web tier, an application tier, and a database tier, wire them together with chained Security Groups, deploy a tiny working application, and then **prove** that the database is unreachable from the internet and from the web tier.

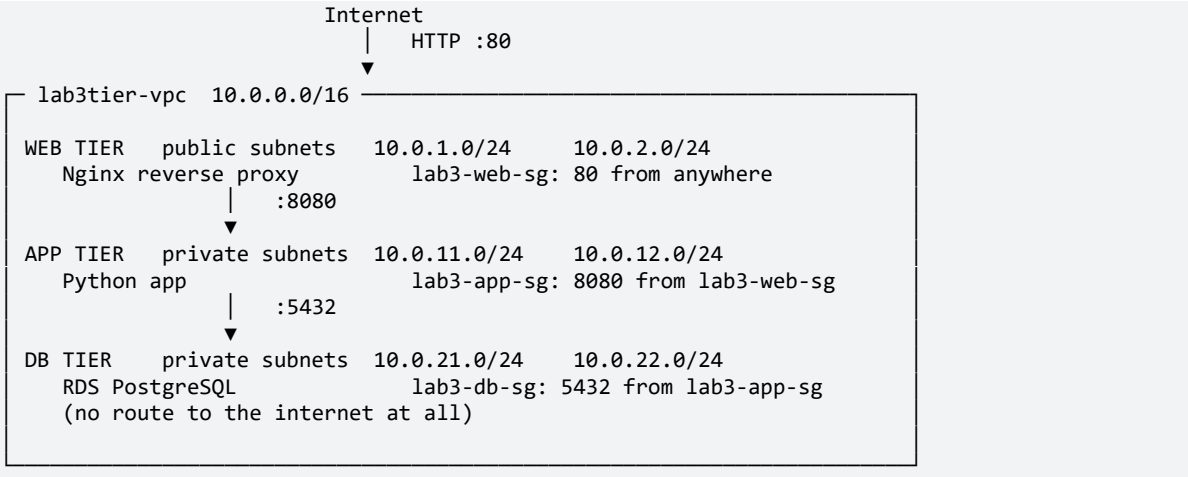
2. The 3-Tier Model in a Nutshell

A **3-tier architecture** splits an application into three layers, each in its own set of subnets with its own firewall rules:

Tier	Job	Subnet type	Reachable from	In this lab
Web (presentation)	Receives user requests and forwards them	Public	The internet	Nginx on EC2
App (logic)	Runs the business logic and talks to the database	Private	The web tier only	Python app on EC2
Data (database)	Stores the data	Private and isolated (no internet route)	The app tier only	RDS PostgreSQL

Why bother? Each layer can be secured and scaled independently, a compromise of the web server doesn't hand an attacker the database, and every tier only accepts traffic from the tier directly above it.

3. Architecture



- The VPC spans **two Availability Zones**, with one subnet per tier in each. RDS requires subnets in at least two AZs for its subnet group, and the second AZ is what lets you add Multi-AZ or more servers later.
- Traffic is allowed only along the chain **Internet → web → app → database**. Each Security Group references the one above it, not an IP range.
- The app tier reaches the internet *outbound only* (for installing packages) through a **NAT Gateway**. The database tier has **no internet route at all**.

4. Prerequisites and Cost

- An AWS account and an SSH client (OpenSSH on macOS, Linux, or Windows 10/11)
- Basic familiarity with VPCs, subnets, route tables, and Security Groups
- About 60–90 minutes, much of it waiting for RDS to become **Available**
- Optional: a local `psql` client, used for one of the isolation tests

> **Cost warning:** unlike a basic free-tier RDS lab, this lab includes resources that are **not free-tier**: a NAT Gateway (billed per hour plus per GB), and public IPv4 addresses (the NAT Gateway's Elastic IP and the web server's public IP). The RDS instance and EC2 instances use small, free-tier-eligible sizes where your account allows it. Do the lab in one sitting and run the clean-up (Step 12) right away. Check the AWS pricing pages for current rates.

5. Lab Tasks (Try These Yourself First!)

1. Create a VPC `lab3tier-vpc` with six subnets across two Availability Zones: web (public), app (private), and database (private) in each AZ.
2. Add an Internet Gateway and a NAT Gateway, and create three route tables so that the web tier is public, the app tier has outbound-only internet access, and the database tier has no internet route.
3. Create three chained Security Groups: web, app, and database.

4. Create a DB subnet group and launch a **non-public** PostgreSQL RDS instance in the database tier.
5. Launch one EC2 instance in the app tier and one in the web tier.
6. Deploy a small Python app on the app server and an Nginx reverse proxy on the web server.
7. Browse to the web server's public IP and confirm that a page counter is being stored in RDS.
8. Prove that the database cannot be reached from the internet or from the web tier.
9. Clean up everything.

Try completing these on your own before checking the solution below.

6. Step-by-Step Solution

Step 1: Create the VPC and Six Subnets

1. Go to **VPC console** → **Your VPCs** → **Create VPC** → **VPC only**.
2. Name tag: `lab3tier-vpc`. IPv4 CIDR: `10.0.0.0/16`. Create.
3. Go to **Subnets** → **Create subnet**, select `lab3tier-vpc`, and add all six subnets (you can add several in one go with **Add new subnet**). Pick **two different Availability Zones** and call them AZ-1 and AZ-2:

Tier	Subnet name	AZ	CIDR
Web (public)	`lab3-web-1`	AZ-1	`10.0.1.0/24`
Web (public)	`lab3-web-2`	AZ-2	`10.0.2.0/24`
App (private)	`lab3-app-1`	AZ-1	`10.0.11.0/24`
App (private)	`lab3-app-2`	AZ-2	`10.0.12.0/24`
Database (private)	`lab3-db-1`	AZ-1	`10.0.21.0/24`
Database (private)	`lab3-db-2`	AZ-2	`10.0.22.0/24`

Right now all six subnets are identical. What makes a subnet "public" or "private" is its **route table**, which you'll set up next.

Step 2: Create the Internet Gateway and the Public Route Table

1. **Internet gateways** → **Create internet gateway** → name `lab3-igw` → Create, then **Actions** → **Attach to VPC** → `lab3tier-vpc`.
2. **Route Tables** → **Create route table**: name `lab3-public-rt`, VPC `lab3tier-vpc`.
3. Select it → **Routes** → **Edit routes** → add `0.0.0.0/0` → **Internet Gateway** → `lab3-igw`. Save.
4. **Subnet associations** → **Edit subnet associations** → tick `lab3-web-1` and `lab3-web-2` → Save.

Step 3: Create the NAT Gateway and the Two Private Route Tables

1. **NAT gateways** → **Create NAT gateway**. Name: `lab3-nat`. Availability mode: **Zonal** (if your console shows this option). Subnet: `lab3-web-1` (it must be a **public** subnet). Connectivity type: **Public**. Click **Allocate Elastic IP**, then **Create NAT gateway**. Wait until it shows **Available**.
2. **App tier route table:** create a route table `lab3-app-rt` in `lab3tier-vpc`. Add the route `0.0.0.0/0` → **NAT Gateway** → `lab3-nat`. Associate it with `lab3-app-1` and `lab3-app-2`.
3. **Database tier route table:** create a route table `lab3-db-rt` in `lab3tier-vpc`. **Add no routes**. It keeps only the built-in `local` route. Associate it with `lab3-db-1` and `lab3-db-2`.

> **Why this matters:** this is the heart of the model. The web tier routes to the Internet Gateway (two-way internet), the app tier routes to the NAT Gateway (outbound only), and the database tier has no way out or in. It can only talk to other addresses inside the VPC.

Step 4: Create the Three Security Groups (in This Order)

Create them in **VPC console** → **Security Groups** → **Create security group**, all in `lab3tier-vpc`. The order matters, because each group refers to the one before it.

Security Group	Inbound rules
`lab3-web-sg`	**HTTP** (80) from `0.0.0.0/0`; **SSH** (22) from **My IP**
`lab3-app-sg`	**Custom TCP** `8080` from **Custom** → select `lab3-web-sg`; **SSH** (22) from **Custom** → select `lab3-web-sg`
`lab3-db-sg`	**PostgreSQL** (5432) from **Custom** → select `lab3-app-sg`

Leave outbound rules at their defaults. Notice that the app and database rules use another Security Group as the *source* instead of an IP range. That means "only resources wearing that group may connect", and it keeps working even when IP addresses change.

Step 5: Create the DB Subnet Group and Launch RDS

Start this early, because RDS takes the longest. You'll build the servers while it provisions.

1. Go to **RDS console** → **Subnet groups** → **Create DB subnet group**.
2. Name: `lab3-db-subnet-group`. Description: `Database tier subnets`. VPC: `lab3tier-vpc`.
3. Availability Zones: select both AZ-1 and AZ-2. Subnets: select only `lab3-db-1` (`10.0.21.0/24`) and `lab3-db-2` (`10.0.22.0/24`). Create.
4. **Databases** → **Create database** → **Standard create**, engine **PostgreSQL**.
5. Templates: **Free tier** (if your account doesn't offer it, choose **Dev/Test** with a small `db.t3.micro`-class instance).
6. Settings: DB instance identifier `lab3-db`, master username `postgres`, and a strong master password. Save the password, because the app server needs it.
7. Connectivity:
   - Compute resource: **Don't connect to an EC2 compute resource**
   - VPC: `lab3tier-vpc`

- DB subnet group: `lab3-db-subnet-group`
- Public access: **No** (this is the big difference from a basic setup with a public database)
- VPC security group: **Choose existing** → select `lab3-db-sg` and remove any default group
- Availability Zone: no preference. Port: `5432`

8. Additional configuration: initial database name `labdb`, backup retention 1 day, and deletion protection **disabled** (so clean-up is easy in a lab).
9. Click **Create database**. It takes several minutes to reach **Available**.

Step 6: Launch the App Server and the Web Server

1. **EC2** → **Key Pairs** → **Create key pair**: name `lab3-key`, RSA, `.pem`. Download it, then on macOS/Linux run `chmod 400 lab3-key.pem`.
2. **Launch the app server: Launch instance**. Name `lab3-app`, AMI **Amazon Linux 2023**, type `t3.micro`, key pair `lab3-key`. Network → **Edit**: VPC `lab3tier-vpc`, Subnet `lab3-app-1`, Auto-assign public IP **Disable**, Security group **Select existing** → `lab3-app-sg`. Launch.
3. **Launch the web server: Launch instance**. Name `lab3-web`, same AMI, type, and key pair. Network → **Edit**: VPC `lab3tier-vpc`, Subnet `lab3-web-1`, Auto-assign public IP **Enable**, Security group `lab3-web-sg`. Launch.
4. Write down the **web server's public IP** and the **app server's private IP** (it will be `10.0.11.x`).

Step 7: Set Up the App Tier

The app server has no public IP, so you reach it by **jumping through the web server** (the web server's Security Group allows SSH from your IP, and the app server's allows SSH from the web server):

```
ssh -i lab3-key.pem -J ec2-user@<WEB_PUBLIC_IP> ec2-user@<APP_PRIVATE_IP>
```

On the app server, install the PostgreSQL driver. This works because the app tier's route table sends outbound traffic through the NAT Gateway:

```
sudo dnf install -y python3-pip
pip3 install --user psycopg2-binary
```

Create a tiny web application that records each page view in the database:

```
cat > app.py <<'EOF'
import os
import psycopg2
from http.server import BaseHTTPRequestHandler, HTTPServer

def get_conn():
    return psycopg2.connect(
        host=os.environ["DB_HOST"], dbname="labdb",
        user=os.environ["DB_USER"], password=os.environ["DB_PASS"],
        connect_timeout=5)

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        status, body = 200, "app tier is up\n"
        if self.path == "/":
            try:
                conn = get_conn()
                cur = conn.cursor()
                cur.execute("CREATE TABLE IF NOT EXISTS visits "
                            "(id SERIAL PRIMARY KEY, "
                            "visited_at TIMESTAMPTZ DEFAULT now(), client TEXT)")
                cur.execute("INSERT INTO visits (client) VALUES (%s)",
                            (self.headers.get("X-Real-IP", self.client_address[0]),))
                cur.execute("SELECT count(*) FROM visits")
                count = cur.fetchone()[0]
                conn.commit()
                conn.close()
                body = ("Hello from the 3-tier lab!\n"
                        f"This page has been viewed {count} times (stored in RDS).\n")
            except Exception as exc:
                status, body = 500, f"Database error: {exc}\n"
        elif self.path != "/health":
            status, body = 404, "not found\n"
        self.send_response(status)
        self.send_header("Content-Type", "text/plain")
        self.end_headers()
        self.wfile.write(body.encode())

HTTPServer(("0.0.0.0", 8080), Handler).serve_forever()
EOF
```

Give the app the database details (copy the **Endpoint** from **RDS console → `lab3-db` → Connectivity & security**) and start it. The `read -s` command asks for the password without echoing it or saving it in your shell history:

```
export DB_HOST=<RDS_ENDPOINT>
export DB_USER=postgres
read -s -p "DB password: " DB_PASS; export DB_PASS; echo

nohup python3 app.py > app.log 2>&1 &

curl http://localhost:8080/health      # app tier is up
curl http://localhost:8080/            # ...viewed 1 times (stored in RDS)
```

If the second `curl` prints `Database error`, check `app.log`, that the RDS status is **Available**, and that the endpoint and password are right. Your app server is now talking to a database that nobody on the internet can reach.

> **For production:** don't pass database passwords in environment variables typed by hand. Use **AWS Secrets Manager** and run the app under a service manager such as `systemd`. This lab keeps it simple.

Step 8: Set Up the Web Tier

Open a new terminal and SSH to the **web server** directly (it has a public IP):

```
ssh -i lab3-key.pem ec2-user@<WEB_PUBLIC_IP>
```

Install Nginx and configure it as a reverse proxy that forwards every request to the app tier:

```
sudo dnf install -y nginx

APP_IP=<APP_PRIVATE_IP>
sudo tee /etc/nginx/nginx.conf > /dev/null <<EOF
user nginx;
pid /run/nginx.pid;
events {}
http {
  server {
    listen 80;
    location / {
      proxy_pass http://$APP_IP:8080;
      proxy_set_header Host \$host;
      proxy_set_header X-Real-IP \$remote_addr;
    }
  }
}
EOF

sudo nginx -t
sudo systemctl enable --now nginx
curl http://localhost/
```

The last command should print the "Hello from the 3-tier lab!" page with a view count of 2. That one request just travelled **Nginx → app server → RDS** and back.

Step 9: Test the Full Path from the Internet

From your **laptop** (or just open the address in a browser):

```
curl http://<WEB_PUBLIC_IP>/
curl http://<WEB_PUBLIC_IP>/
```

Each request shows a higher view count. The number lives in the `visits` table in RDS, so it survives restarting the web or app server. You've just run a real 3-tier application.

Step 10: Prove the Database Is Isolated

Now the important part: show that the layers really are separated. Run each test from the place listed.

#	Run from	Test	Expected result
1	Laptop	Look up and connect to the RDS endpoint	Resolves to a **private** `10.0.21.x`/`10.0.22.x` address and **times out**
2	Laptop	Reach the app server's private IP	**Times out** (private addresses aren't routable from the internet)
3	Web server	Connect to the database port	**BLOCKED** (only the app tier is allowed)
4	App server	Connect to the database port	**OPEN**
5	Web server	Reach the app tier on 8080	Works (the web tier is allowed)

Tests 1 and 2: from your laptop

```
nslookup <RDS_ENDPOINT>
PGCONNECT_TIMEOUT=5 psql --host=<RDS_ENDPOINT> --port=5432 --username=postgres --dbname=labdb
curl -m 5 http://<APP_PRIVATE_IP>:8080/health
```

The lookup returns a private address, and both connection attempts fail. With **Public access: No**, RDS has no public IP at all. (Skip the `psql` line if you don't have it installed.)

Tests 3 and 5: on the web server

```
timeout 5 bash -c '</dev/tcp/<RDS_ENDPOINT>/5432' && echo OPEN || echo BLOCKED
curl http://<APP_PRIVATE_IP>:8080/health
```

The first command prints **BLOCKED**. Even a server inside the same VPC, with a route to the database, is refused, because `lab3-db-sg` only trusts `lab3-app-sg`. The second command works.

Test 4: on the app server

```
timeout 5 bash -c '</dev/tcp/<RDS_ENDPOINT>/5432' && echo OPEN || echo BLOCKED
```

This prints **OPEN**.

> **What this proves:** the database has no public address, the internet can't reach the app tier directly, and even the web tier (which *is* exposed to the internet) can't touch the database. An attacker who breaks into the web server still has to compromise the app server to get anywhere near your data. That's defense in depth.

Step 11 (Optional): Experiments to Deepen Understanding

- **Break the chain:** in `lab3-db-sg`, delete the rule that allows `lab3-app-sg`, then reload the page. It now shows `Database error` after about five seconds. Restore the rule and the page recovers. This shows that the Security Group alone controls database access.
- **Cut the NAT route:** remove the `0.0.0.0/0` route from `lab3-app-rt` and, on the app server, try `curl -I -m 10 https://aws.amazon.com`. It fails, but the page keeps working because the app only needs the database, not the internet. Restore the route afterward.
- **Try Multi-AZ (adds cost): Modify** `lab3-db` to enable a Multi-AZ standby, then use **Actions → Reboot → Reboot with failover**. The page keeps working through the same endpoint. Multi-AZ isn't free-tier, so turn it off or delete the database straight afterward.
- **Add a Network ACL:** create a NACL for the database subnets that only allows TCP 5432 from the two app subnets (plus return traffic on ports 1024–65535), and watch the page keep working. That's a second, subnet-level layer on top of the Security Group.
- **Thought exercise:** what would you add to make the web tier highly available? (Think of an Application Load Balancer, the second-AZ web subnet, and an Auto Scaling group.)

If Something Doesn't Work, Check This List

- **`pip3` or `dnf` hangs on the app server:** does `lab3-app-rt` have `0.0.0.0/0` → NAT Gateway, and is the NAT Gateway in a **public** subnet with state **Available**?
- **Cannot SSH to the app server:** did you jump through the web server (`-J`), and does `lab3-app-sg` allow SSH from `lab3-web-sg`?
- **Nginx shows 502 Bad Gateway:** is the app still running (`curl localhost:8080/health` on the app server), and is `APP_IP` in the Nginx config correct?
- **"Database error" on the page:** is RDS **Available**, does `lab3-db-sg` allow 5432 from `lab3-app-sg`, and are the endpoint and password right (see `app.log`)?
- **The page won't load from the internet:** does `lab3-web-sg` allow HTTP from anywhere, does the web server have a public IP, and are the web subnets associated with `lab3-public-rt`?

- **RDS creation fails on the subnet group:** the DB subnet group needs subnets in **at least two different AZs**.

Step 12: Clean Up (Important — Avoid Charges)

Order matters, because many of these resources depend on each other.

1. **EC2** → terminate `lab3-web` and `lab3-app`.
2. **RDS console** → **Databases** → select `lab3-db` → **Actions → Delete**. Uncheck **Create final snapshot** and **Retain automated backups**, tick the acknowledgement, type `delete me`, and confirm. Wait until it disappears from the list.
3. **RDS console** → **Subnet groups** → delete `lab3-db-subnet-group`.
4. **VPC console** → **NAT gateways** → delete `lab3-nat` and wait for **Deleted**. Then **Elastic IPs** → release the address it used. (A forgotten Elastic IP keeps billing.)
5. **Security Groups** → delete `lab3-db-sg`, then `lab3-app-sg`, then `lab3-web-sg` (in that order, because they reference each other).
6. **Route Tables** → remove the subnet associations and delete `lab3-public-rt`, `lab3-app-rt`, and `lab3-db-rt`.
7. **Internet gateways** → **Actions → Detach from VPC**, then **Delete** `lab3-igw`.
8. **Your VPCs** → select `lab3tier-vpc` → **Actions → Delete VPC**. The console also removes the six subnets.
9. **Key Pairs** → delete `lab3-key` (and the local `.pem` file).
10. Finally, double-check the **RDS**, **EC2**, **NAT gateways**, and **Elastic IPs** pages to confirm that nothing is left running.